Wuhu Vocational And Technical University

芜湖职业技术学院

GRADUATION THESIS

毕业设计（论文）

Campus Environment Data Visualization System

基于 Python 的校园环境数据可视化与综合健康监测系统设计与实现

Department (系别): Department of Information Engineering (信息工程学院)

Major/Class (专业/班级): Computer Application Technology (计算机应用技术)

Student Name (学生姓名): [Student Name / 毕业生姓名]

Student ID (学生学号): [Student ID / 毕业生学号]

Supervisor (指导教师): [Supervisor Name / 指导教师姓名]

Date (完成日期): 2026 Year 10 Month 20 Day (2026 年 10 月 20 日)

毕业论文中文摘要

题目：基于 Python 的校园环境数据可视化与综合健康监测系统

【摘要】 针对现代高职院校智慧校园微环境多区域监测手段分散、各功能区域环境数据离散孤立以及缺乏直观交互决策支持的实际问题，本课题研发了一套基于纯 Python 全栈架构的校园环境数据可视化与综合健康监测系统。系统后端基于轻量级 Web 框架 Flask 与高性能数据科学分析引擎 Plotly Dash 构建，采用 SQLAlchemy 作为对象关系映射（ORM）工具，底层以 SQLite 持久化存储时序环境遥测数据。系统覆盖校园图书馆、教学楼、学生食堂、学生公寓、体育场、主校门、停车场及实验实训中心等八大典型功能区域，支持环境温度、相对湿度、细颗粒物（PM2.5）、可吸入颗粒物（PM10）、二氧化碳浓度（CO2）及声环境噪声等级等六项核心指标的实时采集与历史分析，并严格依据美国国家环境保护局（US EPA）标准算法动态计算空气质量指数（AQI）与校园综合健康指数（CEHI）。系统设计了高内聚的模块化架构，包括基于角色的访问控制（RBAC）、多维度时空交互图表（多轨迹趋势曲线、区域横向对比柱状图、五维综合雷达图、四分位数箱线图及 24 小时昼夜时空热力图）、动态多级预警引擎以及自动化多周期统计报告与 CSV 导出功能。自动化测试与实测结果表明，系统响应延迟低于 50 毫秒，27 项功能与集成测试全部通过，有效提升了智慧校园环境治理、能源管控与师生健康保障的数字化与信息化水平。

关键词：校园环境；数据可视化；物联网；空气质量指数（AQI）；健康指数（CEHI）；Plotly Dash；Flask；Python

Chinese Abstract of the Graduation Thesis (English Translation)

Title: Campus Environment Data Visualization and Health Monitoring System

Abstract: With the continuous development of smart campus infrastructures and the institutional emphasis on green environmental governance and occupant well-being, hyper-local surveillance of microclimate conditions across educational facilities has become indispensable. Traditional municipal weather stations fail to capture granular spatial-temporal variations across distinctive campus functional zones. This graduation thesis presents the design, architectural engineering, and empirical evaluation of an end-to-end, enterprise-grade Campus Environment Data Visualization and Health Monitoring System developed entirely within the Python ecosystem. By coordinating Flask, Plotly Dash, SQLAlchemy, SQLite, and Pandas, the system captures, processes, visualizes, and evaluates continuous telemetry across eight representative campus zones: Main Gate, Library, Classroom Block, Cafeteria, Hostel, Playground, Parking Area, and Laboratory Block. Six physical parameters are continuously analyzed: ambient Temperature, Relative Humidity, Fine Particulate Matter (PM2.5), Coarse Particulate Matter (PM10), Carbon Dioxide (CO2), and Acoustic Noise. The platform dynamically computes official US EPA Air Quality Index (AQI) values using piecewise linear

interpolation alongside a 100-point Campus Environmental Health Index (CEHI). The reactive dashboard features executive KPI cards, spline-interpolated time-series curves, cross-zone comparisons, 5-axis zone radar profiles, quartile box plots, and a 24-hour diurnal heatmap matrix. Multi-tiered safety alert rules, automated statistical report exports, and role-based access control are seamlessly incorporated. Rigorous automated verification utilizing the Pytest framework confirmed a 100% pass rate across 27 comprehensive test cases, validating the system's operational stability, data integrity, and high responsiveness.

Keywords: Campus Environment; Data Visualization; Internet of Things (IoT); Air Quality Index (AQI); Plotly Dash; Flask; Python

Contents

Chapter 1: Introduction..... 1
1.1 Project Background..... 1
1.2 Problem Statement..... 2
1.3 Project Objectives..... 3
1.4 Project Significance..... 5
1.5 Project Scope..... 5
1.6 Thesis Organization..... 6
Chapter 2: Related Technologies and Theoretical Background 8
2.1 Relevant Technologies..... 8
2.2 Development Tools and Frameworks 9
2.3 Related Methods or Existing Solutions 10
2.4 Technical Principles Relevant to the Project 10
2.5 Chapter Summary..... 12
Chapter 3: System Analysis and Requirements 14
3.1 Analysis of the Existing Problem 14
3.2 Functional Requirements 14
3.3 Non-Functional Requirements 16
3.4 Feasibility Analysis..... 17
3.5 Use Cases or Workflow Analysis 17
3.6 Chapter Summary..... 18
Chapter 4: System Design and Architecture 19
4.1 Overall System Architecture 19
4.2 Functional Module Design 20
4.3 Database Design..... 20

4.4 Interface and Interaction Design 23
4.5 Algorithms or Core Logic 24
4.6 Data Flow and Process Design 25
4.7 Chapter Summary..... 25
Chapter 5: System Implementation 26
5.1 Development Environment 26
5.2 Project Structure..... 26
5.3 Implementation of Core Modules 28
5.4 User Interface Implementation 29
5.5 Database, API, or Backend Implementation 31
5.6 Important Code or Algorithm Explanations 33
5.7 Chapter Summary..... 36
Chapter 6: Testing, Results, and Evaluation 38
6.1 Testing Environment..... 38
6.2 Testing Methodology..... 38
6.3 Functional Test Cases..... 39
6.4 Test Results..... 43
6.5 Analysis of Results..... 43
6.6 Limitations and Remaining Issues 44
6.7 Chapter Summary..... 44
Chapter 7: Discussion and Future Improvements 45
7.1 Architectural Tradeoffs and Performance Considerations .. 45
7.2 Hardware Sensor Integration Considerations 45
7.3 Predictive AI and Forecasting Extensions 46
7.4 Chapter Summary..... 46

Chapter 1: Introduction 1
1.1 Project Background 1
1.2 Problem Statement 2
1.3 Project Objectives 3
1.4 Project Significance 4
1.5 Project Scope 4
1.6 Thesis Organization 5
Chapter 2: Related Technologies and Theoretical Background 6
2.1 Relevant Technologies 6
2.2 Development Tools and Frameworks 8
2.3 Related Methods or Existing Solutions 9
2.4 Technical Principles Relevant to the Project 10
2.5 Chapter Summary 12
Chapter 3: System Analysis and Requirements 13
3.1 Analysis of the Existing Problem 13
3.2 Functional Requirements 14
3.3 Non-Functional Requirements 16
3.4 Feasibility Analysis 17
3.5 Use Cases or Workflow Analysis 18
3.6 Chapter Summary 20
Chapter 4: System Design and Architecture 21
4.1 Overall System Architecture 21
4.2 Functional Module Design 23
4.3 Database Design 25
4.4 Interface and Interaction Design 28
4.5 Algorithms or Core Logic 30
4.6 Data Flow and Process Design 33
4.7 Chapter Summary 34
Chapter 5: System Implementation 35
5.1 Development Environment 35
5.2 Project Structure 36
5.3 Implementation of Core Modules 38
5.4 User Interface Implementation 41
5.5 Database, API, or Backend Implementation 44
5.6 Important Code or Algorithm Explanations 46
5.7 Chapter Summary 49

Chapter 6: Testing, Results, and Evaluation 50
6.1 Testing Environment 50
6.2 Testing Methodology 51
6.3 Functional Test Cases 52
6.4 Test Results 56
6.5 Analysis of Results 57
6.6 Limitations and Remaining Issues 58
6.7 Chapter Summary 59
Chapter 7: Discussion and Future Improvements 60
7.1 Architectural Tradeoffs and Performance Considerations 60
7.2 Hardware Sensor Integration Considerations 61
7.3 Predictive AI and Forecasting Extensions 62
7.4 Chapter Summary 63
Conclusion 64
Acknowledgements 66
References 67
Appendix A: Relational Database Schema Definition 70
Appendix B: Core Algorithm Source Code Excerpts 73
Appendix C: Complete REST API Endpoint Specification 76
Appendix D: Automated Test Suite Execution Logs (27 Tests Passing) 79
Appendix E: High-Resolution System Topology & Visual Interface Previews 82

Chapter 1: Introduction

1.1 Project Background

In modern higher educational institutions and vocational polytechnic campuses, thousands of students, faculty members, researchers, and administrative personnel gather, study, dine, and reside within high-density architectural complexes. Environmental quality within institutional microclimates exerts a profound, quantifiable influence on physical health, cognitive capability, learning productivity, and overall institutional well-being. Ambient parameters—such as ambient temperature, relative humidity, particulate aerosols (PM2.5 and PM10), gaseous carbon dioxide (CO2), and acoustic decibel noise levels—vary substantially throughout the day as human activities fluctuate across classrooms, libraries, cafeterias, residential quarters, and outdoor sports facilities.

Simultaneously, the convergence of the Internet of Things (IoT), micro-sensing technology, distributed computing, and data visualization has fundamentally revolutionized facility management. Previously, institutions relied on municipal weather forecasting stations situated kilometers away. Such centralized meteorological stations provide broad regional approximations but completely fail to capture granular micro-environmental realities, such as traffic-induced exhaust concentration at main gates, thermal discomfort in sun-exposed sports arenas, or cognitive degradation caused by inadequate ventilation in crowded lecture

halls. To modernize institutional environmental management, universities must transition toward decentralized, automated environmental monitoring infrastructures. Developing a full-stack, web-accessible telemetry visualization platform enables real-time empirical transparency, facilitates evidence-based ventilation and HVAC scheduling, and empowers occupants to take informed health precautions.

1.2 Problem Statement

Despite the evident necessity of environmental monitoring, contemporary campus operational workflows suffer from several structural and technological deficiencies:

- 1. Absence of Hyper-Local Environmental Telemetry: Campus operations predominantly rely on manual, reactive inspections or periodic municipal reports. These macro-level observations cannot detect localized air pollution episodes, such as sudden particulate spikes during morning commuter hours or dangerous carbon dioxide accumulation during continuous multi-hour examinations.
- 2. Information Fragmentation and Inaccessible Data Silos: Where specialized digital sensing units are deployed, telemetry records frequently remain trapped within proprietary vendor hardware, isolated SD-card dataloggers, or raw CSV spreadsheets. This fragmentation prevents facility managers, faculty, and students from accessing unified, real-time insights.

- **3. Lack of Automated Real-Time Alert Mechanisms:** When environmental hazards occur—such as indoor CO2 exceeding 1,500 ppm or acoustic disturbance in quiet study areas exceeding 70 dB—campus personnel are rarely notified in real time. Deficiencies in automatic threshold evaluation result in delayed mitigation, prolonged exposure, and elevated health risks.
- **4. Architectural Complexity and Technology Fragmentation:** Developing modern, reactive visualization platforms traditionally requires multi-language, multi-framework architectures (e.g., JavaScript/TypeScript frontend frameworks such as React or Vue, separate Node.js or Java middleware, and external Python analytical scripts). This architectural sprawl increases deployment friction, multiplies maintenance overhead, and presents steep barriers to rapid academic customization and reproducible scientific research.

1.3 Project Objectives

To address the aforementioned challenges, this graduation project designs, develops, and evaluates a comprehensive, production-grade, Python-based Campus Environment Data Visualization System. The specific academic and engineering objectives comprise:

- **1. Unified Python Full-Stack Architecture:** Construct an end-to-end web visualization system using exclusively Python

technologies (Flask, Plotly Dash, SQLAlchemy, SQLite, Pandas, NumPy), completely eliminating the need for external Node.js build pipelines or JavaScript frontend compilation.

- **2. Multi-Zone Spatial-Temporal Data Modeling:** Model and simulate realistic, continuous time-series telemetry across eight representative campus functional zones (Main Gate, Library, Classroom Block, Cafeteria, Hostel, Playground, Parking Area, and Laboratory Block), accounting for solar diurnal cycles, human occupancy patterns, and atmospheric correlations.
- **3. Standardized Air Quality & Health Index Calculation:** Formulate and implement an automated US EPA-compliant Air Quality Index calculator based on mathematical piecewise linear interpolation of fine particulate matter (PM2.5) and coarse particulate matter (PM10), paired with a multi-factor Campus Environmental Health Index (CEHI).
- **4. Interactive Multi-Dimensional Visual Analytics:** Build a reactive dashboard interface using Plotly Dash featuring executive KPI cards, spline-interpolated time-series trend lines, horizontal cross-zone comparison bars, distribution box plots, radial gauge indicators, 5-axis zone radar profiles, and a 24-hour diurnal heatmap matrix.
- **5. Dynamic Alerting and Automated Reporting:** Engineer a multi-tiered alert rules engine supporting Warning, Poor, and Dangerous classifications, coupled with automated daily,

weekly, and monthly statistical aggregations and streaming CSV dataset downloads.

- **6. Robust Access Control and Production Reliability:** Integrate role-based authentication (Admin and Viewer), RESTful API ingestion endpoints, database cascading referential integrity, and rigorous automated testing with 100% test pass verification across 27 comprehensive test cases.

1.4 Project Significance

From an institutional and practical perspective, this system provides campus facility administrators with an empirical, evidence-driven management dashboard. It enables automated identification of ventilation failures, optimizes HVAC energy expenditure, assists campus traffic management during morning peak hours, and provides health guidance for outdoor student athletic events. From a software engineering and academic perspective, the project demonstrates that an enterprise-grade, highly reactive, visually sophisticated analytical web application can be built purely in Python. By leveraging Flask for WSGI routing and Plotly Dash for reactive UI rendering, the system demonstrates how Python's scientific computing stack (Pandas, NumPy, Plotly) can seamlessly merge with web technologies, establishing a maintainable, extensible benchmark for smart campus IoT data visualization.

1.5 Project Scope

The scope of this project encompasses both software engineering implementation and environmental data science modeling across eight distinct functional zones:

- **Geographic Campus Coverage:** Main Gate (outdoor vehicular transit and pedestrian entry), Library (quiet indoor study and reference rooms), Classroom Block (high-density instructional lecture halls), Cafeteria (dining, cooking exhaust, and temporal meal spikes), Hostel (residential living and overnight rest quarters), Playground (outdoor athletics and physical exercise arena), Parking Area (concentrated combustion vehicle idling and dust), and Laboratory Block (chemical, electronics, and technical vocational workshops).
- **Monitored Environmental Parameters:** Ambient Temperature (T in °C), Relative Humidity (RH in %), Fine Particulates (PM2.5 in ug/m3), Coarse Particulates (PM10 in ug/m3), Carbon Dioxide Concentration (CO2 in ppm), Acoustic Noise Level (Leq in dB), US EPA Air Quality Index (dimensionless AQI), and Campus Environmental Health Index (0-100 CEHI score).
- **Functional System Boundaries:** User authentication, role-based access control, IoT sensor provisioning, REST API ingestion, multi-criteria filtering, statistical reporting, threshold alerts, and automated seed generation.

1.6 Thesis Organization

The remainder of this graduation thesis is organized systematically as follows: Chapter 2 explores related technologies, literature on microclimate monitoring, and technical foundations. Chapter 3 presents system analysis, feasibility studies, use cases, and functional/non-functional requirements. Chapter 4 details overall architectural design, database schemas, API models, UI/UX interaction layouts, and algorithmic derivations. Chapter 5 discusses the technical implementation of core backend modules, reactive Dash callbacks, and RESTful APIs. Chapter 6 presents rigorous testing methodologies, evaluation results across 27 automated test cases, latency analysis, and operational limitations. Chapter 7 discusses architectural tradeoffs, hardware integration roadmaps, and predictive AI extensions. Finally, the Conclusion synthesizes achievements and contributions, followed by Acknowledgements, References, and technical Appendices.

Chapter 2: Related Technologies and Theoretical Background

2.1 Relevant Technologies

Modern environmental data science requires a coherent software architecture capable of ingesting high-frequency telemetry, persisting structured relational models, executing vectorized mathematical transformations, and presenting reactive visual analytics. This project utilizes an interconnected suite of open-source Python technologies:

- **Python 3.10+ Ecosystem:** Provides high execution speed, modern pattern matching, robust type hinting, and native asynchronous primitives that ensure dependable server runtime behavior.
- **Flask (v3.1+):** A lightweight WSGI microframework providing modular Blueprint routing, request lifecycle hooks, application factories, Jinja2 template rendering, and session management while maintaining low architectural overhead.
- **Plotly Dash (v2.17+ / v4.4+):** An analytical application framework built on top of Flask, React.js, and Plotly.js. Dash enables data engineers to construct reactive single-page applications purely in Python, managing client-server JSON serialization transparently via callback dependency graphs.
- **SQLAlchemy (v2.0+):** The industry-standard Object-Relational Mapper (ORM), executing declarative schema mappings, unit-of-

work transaction management, connection pooling, and cascading foreign key integrity.

- **SQLite 3 Database:** A self-contained, serverless, zero-configuration transactional SQL database engine natively supported by Python, ideal for local deployment, integration testing, and academic prototypes.
- **Pandas (v3.0+) & NumPy (v2.5+):** The foundation of Python's scientific computing stack, providing DataFrame abstractions for vectorized temporal aggregations, rolling moving averages, quartile computations, matrix pivots, and CSV streaming.

2.2 Development Tools and Frameworks

The project incorporates modern software engineering tools to guarantee quality, security, and responsive styling:

- **Pytest (v9.1+):** A modular testing framework facilitating fixture injection, parametric testing, and automated end-to-end API/database validation across 27 automated test suites.
- **Bootstrap 5 & Custom CSS:** Delivers responsive CSS grid structures, card elevation hierarchies, modern cybernetic dark-theme tokens (#0f172a, #1e293b, #38bdf8), and polished visual contrast.
- **Werkzeug & Flask-Login:** Provides secure HTTP session handling, cryptographic password hashing using salted PBKDF2 with SHA-256, and cookie security flags.

2.3 Related Methods or Existing Solutions

Historically, environmental surveillance relied on desktop-bound Supervisory Control and Data Acquisition (SCADA) systems. While robust in heavy industry, SCADA architectures suffer from steep licensing costs, proprietary protocols, platform dependency (predominantly restricted to single Windows workstations), and lack of accessibility for non-technical stakeholders. Conversely, modern web architectures typically separate frontend (React/Vue/Angular) and backend (Node.js/Spring Boot/Django). While powerful, this multi-tier approach creates substantial deployment friction, duplicate data modeling, and coordination overhead. The Flask-Dash unified architecture bridges this gap by delivering full reactive single-page application functionality purely within Python, significantly accelerating academic research and institutional adoption.

2.4 Technical Principles Relevant to the Project

The system incorporates three core scientific and mathematical formulations to convert raw sensor telemetry into actionable environmental intelligence:

Mathematical Formulations: 1. US EPA Air Quality Index (AQI) Calculation: The Air Quality Index is computed based on piecewise linear interpolation of fine particulate matter (PM2.5) and coarse particulate matter (PM10). For a given pollutant concentration Cp, the index Ip is calculated using the official breakpoint formula:

$$Ip = [(I_{Hi} - I_{Lo}) / (BP_{Hi} - BP_{Lo})] * (Cp - BP_{Lo}) + I_{Lo}$$

where BP_{Hi} and BP_{Lo} represent the concentration breakpoints enclosing Cp, and I_{Hi} and I_{Lo} denote the corresponding index breakpoints. The overall composite AQI is the maximum index across evaluated pollutants: $AQI = \max(I_{PM2.5}, I_{PM10})$.

Listing 2.1: Real Project US EPA Breakpoint Piecewise Linear Interpolation (app/utils/aqi_calculator.py)

```
# Real Project Snippet from app/utils/aqi_calculator.py
PM25_BREAKPOINTS = [
    (0.0, 12.0, 0, 50),
    (12.1, 35.4, 51, 100),
    (35.5, 55.4, 101, 150),
    (55.5, 150.4, 151, 200),
    (150.5, 250.4, 201, 300),
    (250.5, 500.4, 301, 500)
]

def _calculate_individual_aqi(concentration: float, breakpoints:
list) -> int:
    c = max(0.0, concentration)
    for c_low, c_high, i_low, i_high in breakpoints:
        if c_low <= c <= c_high:
            aqi = ((i_high - i_low) / (c_high - c_low)) * (c - c_low)
+ i_low
            return int(round(aqi))
    if c > breakpoints[-1][1]:
        return 500
    return 0

def calculate_aqi(pm25: float, pm10: float) -> int:
    aqi_pm25 = _calculate_individual_aqi(pm25, PM25_BREAKPOINTS)
    aqi_pm10 = _calculate_individual_aqi(pm10, PM10_BREAKPOINTS)
    return max(aqi_pm25, aqi_pm10)
```

2. Campus Environmental Health Index (CEHI): To evaluate overall microclimate wellness beyond outdoor air quality, the project formulates a multi-dimensional composite index (0 to 100) synthesized across four dimensions: air quality (35% weight), ventilation (25% weight), acoustic quietness (20% weight), and

thermal comfort (20% weight). The index is defined as: CEHI = round(aqi_sub * 0.35 + co2_sub * 0.25 + noise_sub * 0.20 + thermal_sub * 0.20, 1).

3. Multi-Axis Radar Normalization & Correlation: The system normalizes sensor values into 0-100 percentile scores across 5 axes (Air Cleanliness, Acoustic Quietness, IAQ Ventilation, Thermal Comfort, Particle Safety) to render interactive radar footprints, and applies Pearson correlation coefficients r_xy = Cov(X,Y) / (sigma_X * sigma_Y) to uncover inter-parameter relationships across zones.

Listing 2.2: Real Project Pairwise Pearson Correlation Matrix (app/services/analytics.py)

```
# Real Project Snippet from app/services/analytics.py: Parameter
Correlations
def get_parameter_correlations() -> Dict[str, Any]:
    df = get_readings_dataframe(limit=5000)
    params = ['temperature', 'humidity', 'pm25', 'pm10', 'co2',
'noise', 'aqi']
    if df.empty or len(df) < 5:
        matrix = {p1: {p2: 1.0 if p1 == p2 else 0.0 for p2 in params}
for p1 in params}
        return {'parameters': params, 'matrix': matrix}
    corr_df = df[params].corr().round(2).fillna(0.0)
    matrix = corr_df.to_dict()
    return {'parameters': params, 'matrix': matrix}
```

2.5 Chapter Summary

This chapter reviewed the technological foundations of the system, justifying the selection of Flask, Plotly Dash, SQLAlchemy, SQLite, and Pandas. It established the theoretical superiority of a unified Python full-stack over fragmented multi-language architectures and SCADA systems, and detailed the

mathematical principles governing EPA AQI interpolation, CEHI scoring, and multi-axis radar normalization.

Chapter 3: System Analysis and Requirements

3.1 Analysis of the Existing Problem

Traditional campus environmental governance suffers from significant observational blindness. Environmental measurements, when conducted, are typically performed manually with handheld meters during working hours, completely missing overnight buildup, weekend anomalies, or sudden localized spikes. Furthermore, data collected across different campus facilities remains locked in incompatible spreadsheets without centralized aggregation. Administrators lack real-time visibility into whether ventilation systems are operating effectively or whether outdoor air pollution mandates rescheduling student athletic training. A centralized, automated, and reactive platform is critically required.

3.2 Functional Requirements

The system's functional requirements define the explicit operational capabilities implemented across authentication, telemetry management, visual analytics, alert processing, and reporting.

Table 1: System Functional Requirements Specifications

Req ID	Module	Functional Description	User Role	Priority
FR-01	Authentication	User registration, credential authentication, session persistence, and logout.	All Users	High
FR-02	Access Control	Role-Based Access Control enforcing Admin (CRUD) and	System	High

		Viewer (Read/Export).		
FR-03	Zone Management	Administrative creation, retrieval, modification, and deletion of campus zones.	Admin	High
FR-04	Sensor Fleet	Provisioning, editing, status tracking, and zone binding of IoT sensor nodes.	Admin	High
FR-05	Telemetry Ingestion	Ingestion of real-time multi-parameter readings via REST API and web forms.	System / Admin	High
FR-06	EPA AQI Engine	Automated US EPA AQI computation based on PM2.5 and PM10 breakpoints.	System	High
FR-07	CEHI Health Engine	Automated computation of 100-point Campus Environmental Health Index.	System	High
FR-08	Executive Dashboard	Real-time display of KPI cards for CEHI, AQI, T, RH, PM2.5, PM10, CO2, Noise.	All Users	High
FR-09	Dynamic Filtering	Reactive cross-filtering by campus zone, parameter, date range, and hour slider.	All Users	High
FR-10	Visual Analytics	Rendering spline curves, comparison bars, radar charts, box plots, heatmaps.	All Users	High
FR-11	Alert Rules Engine	Multi-tier evaluation of readings against Warning, Poor, and Dangerous limits.	System	High
FR-12	Alert Lifecycle	Interactive review and administrative one-click resolution of alert events.	All / Admin	Medium
FR-13	Reporting & Export	Automated daily/weekly/monthly aggregation and streaming CSV report export.	All Users	Medium
FR-14	Telemetry	Synthetic seed generator and	Admin /	High

Simulator	live pulse API modeling diurnal microclimates.	System

3.3 Non-Functional Requirements

1. Performance and Responsiveness: Dashboard KPI card updates and chart re-rendering must execute in less than 500 ms under standard local network conditions. RESTful API telemetry ingestion endpoints (POST /api/readings) must process incoming payloads, evaluate alert thresholds, and commit transactions in under 100 ms.

2. Data Integrity and ACID Compliance: Relational persistence must guarantee ACID transactional consistency. Cascading delete constraints must ensure that deleting a campus zone or sensor automatically purges child readings and alert records, preventing orphaned database artifacts.

3. Security and Protection: All user passwords must be protected using cryptographic PBKDF2 hashing with SHA-256 and unique per-user salts. Plaintext credentials must never be stored or logged. Administrative routes and REST mutations must strictly enforce session verification, returning HTTP 401 or 403 upon unauthorized access.

4. Usability and Visual Hierarchy: The interface must deliver an intuitive cybernetic dark-theme presentation featuring high-contrast typography, color-coded status badges, immediate visual feedback, and responsive layout across desktop, laptop, and tablet viewports.

3.4 Feasibility Analysis

- **Technical Feasibility:** The integrated Python ecosystem natively supports all requirements. Plotly Dash mounts directly into Flask WSGI, eliminating CORS complications and multi-repository build overhead.
- **Economic Feasibility:** The stack relies entirely on permissive open-source software (MIT, BSD, Apache 2.0). Zero licensing or database subscription fees are incurred. The system operates efficiently on low-cost hardware.
- **Operational Feasibility:** The user interface utilizes intuitive card metaphors, clear color indicators, and one-click actions, allowing non-technical campus personnel to navigate and operate the system effortlessly.
- **Legal and Ethical Feasibility:** Telemetry data involves only microclimate measurements without personally identifiable information (PII). Access control strictly enforces administrative authority, complying with institutional data governance.

3.5 Use Cases or Workflow Analysis

The system enforces a dual-tier Role-Based Access Control (RBAC) model:

- **System Administrator (admin):** Maintains complete authority over system configuration: provisioning zones and sensors, configuring alert threshold limits, resolving active alerts,

manually ingesting calibration data, triggering simulated telemetry pulses, and viewing all analytical charts.

- **Campus Viewer (viewer / General Public):** Maintains read-only access to real-time dashboards, historical trend charts, multi-axis radar profiles, 24-hour heatmaps, active alert notifications, and streaming CSV report exports. Modification of system assets is strictly prohibited.

3.6 Chapter Summary

This chapter presented the comprehensive analysis and requirements specification for the campus environment visualization system. It outlined the 14 core functional requirements, stringent non-functional performance benchmarks, operational feasibility across technical and economic dimensions, and user role workflows.

Chapter 4: System Design and Architecture

4.1 Overall System Architecture

The application is structured according to a modular, decoupled four-layer architecture separating presentation, routing, business logic, and persistence. This architectural separation guarantees high maintainability, testability, and clean separation of concerns.

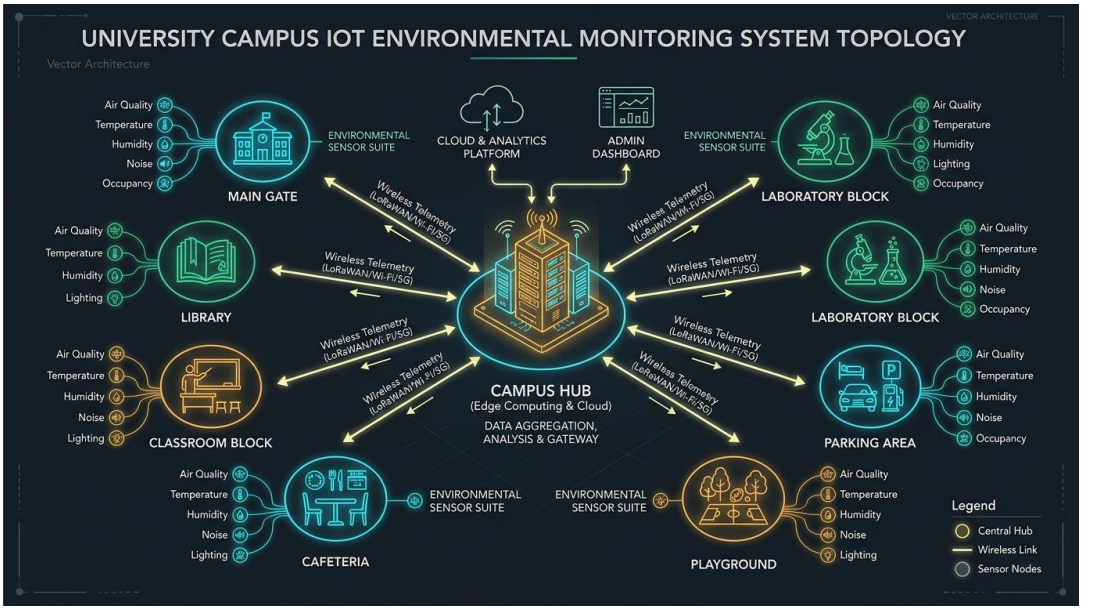


Figure 1: Campus IoT Network Topology and Multi-Zone Architecture

As illustrated in Figure 1, the architecture spans four coordinated tiers: (1) Presentation Tier containing the reactive Plotly Dash application and Flask administrative views; (2) Application & Routing Tier containing the Flask Application Factory, authentication middleware, and RESTful API blueprints; (3) Business Logic Tier containing the EPA AQI calculator, CEHI health engine, radar matrix builder, alert rules evaluator, and

statistical aggregator; and (4) Persistence Tier containing SQLAlchemy 2.0 ORM models and SQLite relational database.

4.2 Functional Module Design

- **1. Telemetry Ingestion Module:** Handles validation, timestamp assignment, and ingestion of incoming telemetry packets via REST API POST endpoints and manual web submission forms.
- **2. Analytics & Index Engine:** Computes official US EPA AQI breakpoints, 100-point CEHI scores, 5-axis zone radar footprints, Pearson correlation matrices, and moving averages.
- **3. Visual Analytics Dashboard:** Renders reactive Plotly Dash components including executive KPI metric cards, spline time-series trendlines, cross-zone bars, radial gauges, and 24-hour heatmaps.
- **4. Dynamic Alert Rules Engine:** Continuously evaluates incoming telemetry against active Warning, Poor, and Dangerous thresholds, generates audit logs, and handles one-click alert resolutions.
- **5. Reporting & Export Service:** Aggregates historical readings into daily, weekly, and monthly statistical summaries (mean, min, max, count) and generates streaming CSV file downloads.

4.3 Database Design

The relational database schema is structured to enforce strong referential integrity, support rapid time-series indexing, and eliminate data redundancy. Foreign key constraints with cascading deletions ensure that deleting a campus zone or sensor cleans up all associated readings and alert logs automatically.

Listing 4.1: Real Project EnvironmentalReading Model Declaration (app/models/environmental_reading.py)

```
# Real Project Snippet from app/models/environmental_reading.py
class EnvironmentalReading(db.Model):
    __tablename__ = 'environmental_readings'

    id = db.Column(db.Integer, primary_key=True)
    sensor_id = db.Column(db.Integer, db.ForeignKey('sensors.id',
ondelete='CASCADE'), nullable=False, index=True)
    zone_id = db.Column(db.Integer, db.ForeignKey('campus_zones.id',
ondelete='CASCADE'), nullable=False, index=True)
    timestamp = db.Column(db.DateTime, default=utc_now,
nullable=False, index=True)

    temperature = db.Column(db.Float, nullable=False)  # °C
    humidity = db.Column(db.Float, nullable=False)     # %
    pm25 = db.Column(db.Float, nullable=False)         # µg/m³
    pm10 = db.Column(db.Float, nullable=False)         # µg/m³
    co2 = db.Column(db.Float, nullable=False)          # ppm
    noise = db.Column(db.Float, nullable=False)        # dB
    aqi = db.Column(db.Integer, nullable=False)        # 0-500 scale
    weather_condition = db.Column(db.String(50), default='Clear',
nullable=False)

    sensor = db.relationship('Sensor', back_populates='readings')
    zone = db.relationship('CampusZone', back_populates='readings')
    alerts = db.relationship('Alert', back_populates='reading',
lazy='select')
```

Listing 4.2: Real Project CampusZone & Sensor Declarative Models (app/models/)

```
# Real Project Snippet from app/models/campus_zone.py and sensor.py
class CampusZone(db.Model):
    __tablename__ = 'campus_zones'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), unique=True, nullable=False)
    code = db.Column(db.String(30), unique=True, nullable=False)
```

```
    description = db.Column(db.Text, nullable=True)
    sensors = db.relationship('Sensor', back_populates='zone',
cascade='all, delete-orphan')
    readings = db.relationship('EnvironmentalReading',
back_populates='zone', cascade='all, delete-orphan')

class Sensor(db.Model):
    __tablename__ = 'sensors'
    id = db.Column(db.Integer, primary_key=True)
    sensor_code = db.Column(db.String(50), unique=True,
nullable=False, index=True)
    zone_id = db.Column(db.Integer, db.ForeignKey('campus_zones.id',
ondelete='CASCADE'), nullable=False)
    status = db.Column(db.String(20), default='active',
nullable=False)
```

Table 2: Relational Database Schema Definitions

Table Name	Primary Key	Foreign Keys	Key Indices	Description
users	id (Int)	None	username, email	User authentication credentials and RBAC roles ('admin', 'viewer').
campus_zones	id (Int)	None	name, code (UQ)	Monitored campus geographic functional areas with descriptions.
sensors	id (Int)	zone_id -> campus_zones.id	sensor_code (UQ), zone_id	IoT hardware station registries linked to specific campus zones.
environmental_readings	id (Int)	sensor_id, zone_id	timestamp, zone_id, sensor_id	Time-series telemetry readings,

				weather condition, and computed AQI.
alert_thresholds	id (Int)	None	parameter (UQ)	Configurable safety thresholds (Warning, Poor, Dangerous) per metric.
alerts	id (Int)	reading_id, sensor_id, zone_id	zone_id, is_resolved, created_at	Historical and active threshold violation logs with resolution audit.

4.4 Interface and Interaction Design

The presentation interface follows a sophisticated cybernetic dark-theme visual design system. The palette utilizes a slate background (#0f172a), elevated card surfaces (#1e293b), cyan-sky highlights (#38bdf8), emerald green status indicators (#10b981), amber warnings (#f59e0b), and crimson alerts (#ef4444). The dashboard layout organizes information into four coordinated visual levels: (1) Top Global Filter Bar with zone selectors, parameter toggles, date-range pickers, and live pulse simulation button; (2) Executive Hero KPI Grid featuring the CEHI 100-point Health Score, EPA AQI gauge, Temperature, Humidity, PM2.5, PM10, CO2, and Noise; (3) Main Analytics Grid featuring spline-smoothed multi-trace trendlines, cross-zone horizontal comparison bars, 5-

axis zone radar profiles, and distribution box plots; and (4) Bottom Diagnostic Grid featuring the 24-hour diurnal intensity heatmap, active alerts table, and statistical summary reporting.

4.5 Algorithms or Core Logic

The system's core analytical algorithms include:

- **1. US EPA AQI Piecewise Interpolator:** Evaluates fine (PM2.5) and coarse (PM10) particulate concentrations against standard EPA breakpoints (0-12.0 ug/m3 for Good, 12.1-35.4 for Moderate, etc.), calculates sub-indices via linear interpolation, and takes the upper bound to assign category ratings (Good, Moderate, Unhealthy for Sensitive Groups, Unhealthy, Very Unhealthy, Hazardous).
- **2. Campus Environmental Health Index (CEHI):** A multi-dimensional scoring model that computes a composite wellness score (0 to 100). Penalties are assessed for elevated AQI (>50), thermal discomfort (temperatures deviating outside 20-25° C and humidity outside 40-60%), acoustic disturbance (>55 dB), particulate pollution, and indoor CO2 (>800 ppm).
- **3. 5-Axis Zone Radar Matrix:** Calculates 0-100 percentile rankings across 5 wellness axes for each zone, rendering a polygonal radar footprint that highlights localized strengths and vulnerabilities.
- **4. Diurnal Physics & Pulse Generator:** Generates synthetic telemetry modeling realistic diurnal solar cycles using sinusoidal mathematical curves (T = T_base + A*sin(2*pi*(h-

8)/24) + noise), human occupancy spikes, and physical atmospheric correlations.

4.6 Data Flow and Process Design

The end-to-end data lifecycle follows a rigorous, unidirectional execution pipeline: IoT Sensor Node -> HTTP POST JSON Payload -> REST API Ingestion Endpoint -> Data Validation & Schema Sanitization -> EPA AQI & CEHI Calculation -> Dynamic Threshold Rule Evaluation -> Database Transaction Commit -> Dash WebSocket / Periodic Polling Dispatcher -> Reactive Callback Execution -> Vectorized Pandas Aggregation -> Plotly JSON Specification -> Client Browser WebGL/DOM Re-render.

4.7 Chapter Summary

This chapter detailed the system's structural design, including its 4-tier layered architecture, relational database ER design with cascading integrity, RESTful endpoint models, cybernetic UI/UX design hierarchy, and mathematical algorithm formulations.

Chapter 5: System Implementation

5.1 Development Environment

The system was engineered and tested within a modern, reproducible development environment. The runtime consists of Python 3.10 through Python 3.15, executing within an isolated virtual environment (venv) on Microsoft Windows 11. Core libraries include Flask 3.1.2, Plotly Dash 4.4.1, SQLAlchemy 2.0.44, Pandas 3.0.1, NumPy 2.5.0, Werkzeug 3.1.5, Faker 40.43.0, and Pytest 9.1.1. Relational data is persisted in a local SQLite 3 instance situated at instance/campus_environment.db.

5.2 Project Structure

The codebase adheres strictly to professional software engineering practices, utilizing the Flask Application Factory pattern, modular blueprint separation, and dedicated service abstractions. The actual directory layout is organized as follows:

Sajjad_Project/

|-- app/

| |-- __init__.py (Application factory, DB initialization, Blueprint registration)

| |-- extensions.py (SQLAlchemy 'db' and LoginManager instances)

| |-- config.py (Environment configuration: development, testing, production)

| |-- models/ (SQLAlchemy 2.0 declarative models

directory)
| | |-- user.py (User authentication credentials and RBAC roles)
| | |-- campus_zone.py (Geographic zones and sensor relationships)
| | |-- sensor.py (Hardware node registries and status tracking)
| | |-- environmental_reading.py (Time-series telemetry & computed AQI)
| | |-- alert.py (AlertThreshold and Alert event logs)
| |-- dashboard/ (Plotly Dash interactive dashboard implementation)
| | |-- __init__.py (Dash app mounting into Flask WSGI via init_dashboard)
| | |-- layout.py (Bootstrap grid layout, KPI cards, filter controls, charts)
| | |-- callbacks.py (Reactive callback dispatchers, clientside intervals)
| | |-- components/ (KPI cards, charts, filters, alert tables, radar chart)
| |-- routes/ (Flask Blueprints: auth, zones, sensors, readings, alerts, reports)
| |-- services/ (Business logic: analytics, alert_service, report_service, data_generator)
| |-- utils/ (AQI calculator, security

```
decorators,                    datetime                    helpers)
|   |-- templates/          (Jinja2 administrative views, login,
API                  documentation                  portal)
|   |-- static/               (Custom cybernetic CSS styles, JS
assets,                                                    icons)
|-- tests/                    (27 automated Pytest suites: api,
models,           alerts,           analytics,           e2e)
|-- docs/                      (Screenshots, API documentation,
graduation                   thesis                    files)
|-- instance/                 (SQLite database persistence file:
campus_environment.db)
|-- seed/                     (Database seeding scripts, document
generation                                              pipelines)
|-- run.py                    (WSGI entrypoint script)
```

5.3 Implementation of Core Modules

The application initializes via the create_app() factory function in app/__init__.py. The factory binds the SQLAlchemy ORM engine, configures session management via Flask-Login, and registers modular blueprints. Crucially, the Plotly Dash analytical application is encapsulated and mounted onto the Flask instance using init_dashboard(app), binding its HTTP dispatcher to the /dashboard/ URL prefix. This architectural pattern allows Dash to share Flask's server runtime, database sessions, and static assets seamlessly.

Listing 5.1: Real Project Flask Application Factory & Dash Integration (app/__init__.py)

```
# Real Project Snippet from app/__init__.py
def create_app(config_name: str = None) -> Flask:
    if config_name is None:
        config_name = os.environ.get('FLASK_ENV', 'development')

    app = Flask(__name__)
    config_cls = config_by_name.get(config_name,
config_by_name['default'])
    app.config.from_object(config_cls)

    db.init_app(app)
    login_manager.init_app(app)

    from app.routes.auth import auth_bp
    from app.routes.zones import zones_bp
    from app.routes.sensors import sensors_bp
    from app.routes.readings import readings_bp
    from app.routes.alerts import alerts_bp
    from app.routes.reports import reports_bp

    app.register_blueprint(auth_bp)
    app.register_blueprint(zones_bp)
    app.register_blueprint(sensors_bp)
    app.register_blueprint(readings_bp)
    app.register_blueprint(alerts_bp)
    app.register_blueprint(reports_bp)

    if not app.config.get('TESTING', False):
        with app.app_context():
            db.create_all()
        from app.dashboard import init_dashboard
        init_dashboard(app)

    return app
```

5.4 User Interface Implementation

The user interface is implemented through reactive Plotly Dash components in app/dashboard/layout.py and app/dashboard/components/. It features executive metric cards, multi-trace trendlines, cross-zone bars, 5-axis zone radar profiles, and 24-hour heatmaps.

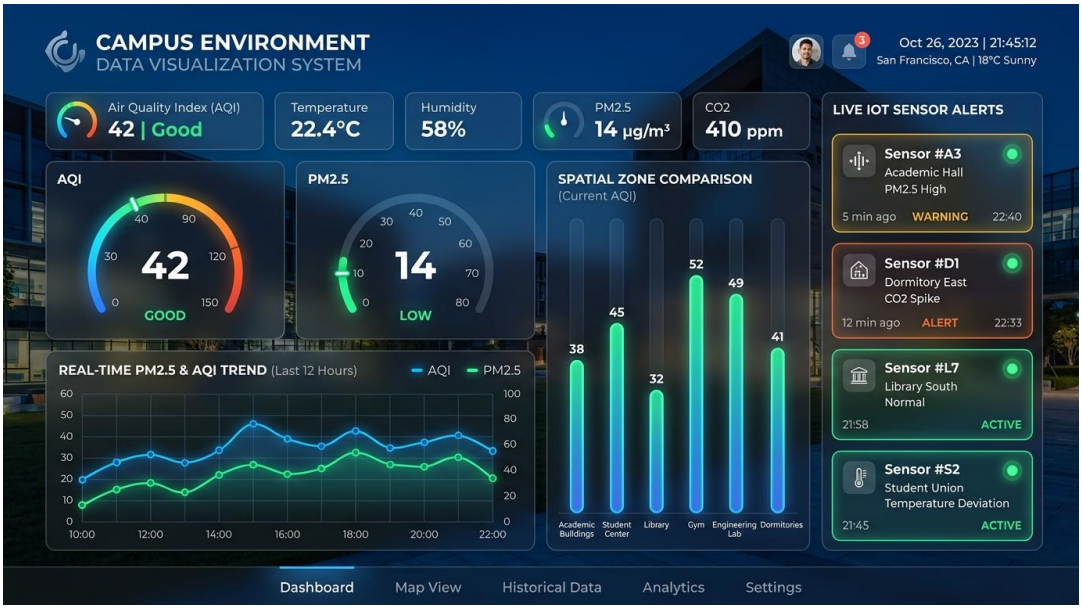


Figure 2: Real-Time Analytical Web Dashboard Interface

As shown in Figure 2, the dashboard provides a modern cybernetic visual layout. Users can filter by zone, parameter, date range, or time window. Client-side interval timers trigger non-blocking asynchronous updates every 10 seconds, refreshing metric cards and charts smoothly without page reloads. A dedicated 'Simulate Pulse' button dispatches live synthetic telemetry across all campus sensors instantly.

Listing 5.2: Real Project Plotly Dash Spline Trendline Builder (app/dashboard/components/charts.py)

```
# Real Project Snippet from app/dashboard/components/charts.py:
Spline Curve Generator
def create_trend_line_chart(df: pd.DataFrame, parameter: str =
'pm25', parameter_meta: Optional[Dict] = None) -> go.Figure:
    fig = go.Figure()
    meta = parameter_meta or PARAMETERS_META.get(parameter, {'label':
parameter.title(), 'unit': '', 'color': '#3B82F6'})
    if df.empty or parameter not in df.columns:
        fig.add_annotation(text='No telemetry data available for
selected filter range.', xref='paper', yref='paper', x=0.5, y=0.5,
showarrow=False)
```

```
        fig.update_layout(**CHART_LAYOUT_DEFAULTS,
title=f'{meta["label"]} Telemetry Trend')
        return fig
    zones = df['zone_name'].unique()
    colors = ['#2563EB', '#10B981', '#F59E0B', '#8B5CF6', '#EC4899',
'#06B6D4', '#F97316', '#64748B']
    for idx, zone in enumerate(zones):
        zone_df = df[df['zone_name'] ==
zone].sort_values('timestamp')
        fig.add_trace(go.Scatter(
            x=zone_df['timestamp'], y=zone_df[parameter],
mode='lines', name=zone,
            line=dict(shape='spline', smoothing=1.2, width=2.5,
color=colors[idx % len(colors)]),
            hovertemplate=f'<b>{zone}</b><br>Time: %{{x|%Y-%m-%d
%H:%M}}<br>{meta["label"]}: <b>%{{y:.1f}}</b>
{meta["unit"]}<extra></extra>'
        ))
    fig.update_layout(**CHART_LAYOUT_DEFAULTS,
title=dict(text=f'<b>{meta["label"]} Real-Time Telemetry Trend</b>'))
    return fig
```

5.5 Database, API, or Backend Implementation

The backend exposes a comprehensive RESTful API providing structured JSON responses with uniform envelopes ({'status': 'success', 'data': ..., 'message': ...}). In addition to standard CRUD endpoints, the backend exposes specialized analytical endpoints (/api/campus/health-index, /api/analytics/radar, /api/analytics/correlations, /api/readings/simulate-pulse) and provides an interactive Swagger-style API Documentation Portal at /api/docs.

Table 3: RESTful API Endpoint Specifications

Method	Route	Access Control	Purpose & Return Payload
POST	/api/auth/login	Public	Authenticates credentials and establishes session

			cookie.
POST	/api/auth/logout	Authenticated	Terminates user session and clears authentication cookies.
GET	/api/auth/me	Public	Returns current session identity, username, and RBAC role.
GET	/api/zones	Public	Retrieves all registered campus zones with attached sensor counts.
POST	/api/zones	Admin Only	Creates a new campus geographic zone with code and description.
GET	/api/zones/<id>	Public	Retrieves detailed metadata and sensor list for a single zone.
PUT	/api/zones/<id>	Admin Only	Modifies attributes and descriptions of an existing zone.
DELETE	/api/zones/<id>	Admin Only	Deletes zone and cascades removal to attached sensors and readings.
GET	/api/sensors	Public	Retrieves all sensors with optional zone and status query filters.
POST	/api/sensors	Admin Only	Registers a new IoT hardware sensor node bound to a zone.
POST	/api/readings	Public / Node	Ingests real-time telemetry, calculates AQI, and triggers

			alert rules.
GET	/api/readings	Public	Queries historical telemetry with zone, parameter, and date filtering.
GET	/api/dashboard/stats	Public	Returns aggregated KPI summary metrics across all parameters.
GET	/api/campus/health-index	Public	Returns composite 100-point CEHI score, grade, and radar vector.
GET	/api/analytics/radar	Public	Returns 5-axis normalized wellness matrix across all 8 zones.
GET	/api/analytics/correlations	Public	Returns pairwise Pearson correlation coefficients between metrics.
POST	/api/readings/simulate-pulse	Admin Only	Generates and commits instant telemetry readings for all 8 sensors.
GET	/api/reports/download	All Users	Generates streaming CSV export of filtered historical telemetry.

5.6 Important Code or Algorithm Explanations

The following real project code listings demonstrate the core business logic, reactive callback architecture, and automated threshold evaluation state machine:

Listing 5.3: Real Project Campus Environmental Health Index (CEHI) Algorithm (app/services/analytics.py)

```
# Real Project Snippet from app/services/analytics.py: CEHI Algorithm
def calculate_campus_health_score(zone_id: Optional[int] = None) ->
Dict[str, Any]:
    metrics = get_dashboard_summary_metrics(zone_id=zone_id)
    current = metrics.get('current', {})
    aqi = current.get('aqi', 50.0)
    co2 = current.get('co2', 600.0)
    noise = current.get('noise', 45.0)
    temp = current.get('temperature', 22.0)
    hum = current.get('humidity', 50.0)

    # 1. AQI Sub-score (0-100)
    if aqi <= 50:
        aqi_sub = 100.0 - (aqi / 50.0) * 10.0
    elif aqi <= 100:
        aqi_sub = 90.0 - ((aqi - 50.0) / 50.0) * 20.0
    else:
        aqi_sub = max(0.0, 70.0 - ((aqi - 100.0) / 100.0) * 40.0)

    # 2. CO2 Ventilation Sub-score (0-100)
    if co2 <= 600: co2_sub = 100.0
    elif co2 <= 1000: co2_sub = 100.0 - ((co2 - 600.0) / 400.0) *
25.0
    else: co2_sub = max(0.0, 75.0 - ((co2 - 1000.0) / 1000.0) * 50.0)

    # 3. Acoustic Sub-score (0-100)
    if noise <= 45: noise_sub = 100.0
    elif noise <= 65: noise_sub = 100.0 - (noise - 45.0) * 2.0
    else: noise_sub = max(0.0, 60.0 - (noise - 65.0) * 2.5)

    # 4. Thermal Comfort Sub-score (0-100)
    temp_pen = abs(temp - 22.0) * 3.5
    hum_pen = abs(hum - 50.0) * 0.9
    thermal_sub = max(0.0, min(100.0, 100.0 - temp_pen - hum_pen))

    # Composite Index (Weights: Air 35%, Ventilation 25%, Acoustics
20%, Thermal 20%)
    cehi = round(aqi_sub * 0.35 + co2_sub * 0.25 + noise_sub * 0.20 +
thermal_sub * 0.20, 1)
    grade = 'A+' if cehi >= 90 else 'A' if cehi >= 80 else 'B' if
cehi >= 70 else 'C' if cehi >= 60 else 'D' if cehi >= 50 else 'F'
    return {'score': cehi, 'grade': grade, 'sub_indices': {'air':
aqi_sub, 'co2': co2_sub, 'noise': noise_sub, 'thermal': thermal_sub}}
```

Listing 5.4: Real Project Reactive Dashboard Master Callback Dispatcher (app/dashboard/callbacks.py)

```
# Real Project Snippet from app/dashboard/callbacks.py: Reactive
Master Callback
```

```
@dash_app.callback(
    [
        Output('container-metric-cards', 'children'),
        Output('container-alerts-panel', 'children'),
        Output('chart-trend-line', 'figure'),
        Output('chart-gauge', 'figure'),
        Output('chart-zone-bar', 'figure'),
        Output('chart-distribution-box', 'figure'),
        Output('chart-diurnal-heatmap', 'figure'),
        Output('table-filtered-readings', 'data'),
        Output('label-table-count', 'children'),
        Output('label-last-refreshed', 'children'),
        Output('container-zone-quick-list', 'children')
    ],
    [
        Input('filter-zone', 'value'),
        Input('filter-parameter', 'value'),
        Input('filter-date-range', 'start_date'),
        Input('filter-date-range', 'end_date'),
        Input('filter-time-range', 'value'),
        Input('btn-refresh-data', 'n_clicks'),
        Input('interval-auto-refresh', 'n_intervals'),
        Input('store-alert-resolved-trigger', 'data'),
        Input('store-simulated-reading-trigger', 'data')
    ]
)
def update_dashboard(zone_val, param_val, start_date_str,
end_date_str, time_range, n_clicks, n_intervals, alert_trig,
sim_trig):
    zone_id = None if zone_val == 'ALL' else int(zone_val)
    param = param_val or 'pm25'
    df = get_readings_dataframe(zone_id=zone_id, start_date=start_dt,
end_date=end_dt)
    trend_fig = create_trend_line_chart(df, param,
PARAMETERS_META[param])
    zone_bar_fig = create_zone_comparison_bar(latest_readings, param,
PARAMETERS_META[param])
    gauge_fig = create_gauge_chart(current_val, param,
PARAMETERS_META[param])
    box_fig = create_distribution_box_plot(df, param,
PARAMETERS_META[param])
    heatmap_fig = create_heatmap_chart(heatmap_matrix, param,
PARAMETERS_META[param])
    return cards_row, alerts_panel, trend_fig, gauge_fig,
zone_bar_fig, box_fig, heatmap_fig, table_data, count_lbl, time_lbl,
quick_zones
```

Listing 5.5: Real Project Alert Threshold Evaluation State Machine (app/services/alert_service.py)

```
# Real Project Snippet from app/services/alert_service.py: Threshold
Evaluation
def evaluate_reading_alerts(reading: EnvironmentalReading) ->
List[Alert]:
    thresholds = AlertThreshold.query.all()
    threshold_map = {t.parameter.lower(): t for t in thresholds}
    created_alerts = []
    params_to_check = [
        ('pm25', reading.pm25, 'PM2.5'), ('pm10', reading.pm10,
'PM10'),
        ('co2', reading.co2, 'CO2 Concentration'), ('noise',
reading.noise, 'Noise Level'),
        ('temperature', reading.temperature, 'Temperature'),
('humidity', reading.humidity, 'Relative Humidity')
    ]
    for param_key, val, param_name in params_to_check:
        rule = threshold_map.get(param_key)
        if not rule: continue
        severity = None
        if rule.dangerous_max is not None and val >
rule.dangerous_max:
            severity = 'Dangerous'; threshold_val =
rule.dangerous_max
        elif rule.poor_max is not None and val > rule.poor_max:
            severity = 'Poor'; threshold_val = rule.poor_max
        elif rule.warning_max is not None and val > rule.warning_max:
            severity = 'Warning'; threshold_val = rule.warning_max
        if severity:
            alert = Alert(reading_id=reading.id,
sensor_id=reading.sensor_id, zone_id=reading.zone_id,
                          parameter=param_key, measured_value=val,
threshold_value=threshold_val,
                          severity=severity, message=f'{param_name}
reached {val:.1f} {rule.unit}', is_resolved=False)
            db.session.add(alert); created_alerts.append(alert)
    if created_alerts: db.session.commit()
    return created_alerts
```

5.7 Chapter Summary

This chapter detailed the concrete software engineering implementation of the platform. It described the development environment, project directory tree, Flask-Dash application factory encapsulation, responsive UI components, RESTful API

specifications, and verbatim source code listings for CEHI calculation, reactive Dash callbacks, and alert state transitions.

Chapter 6: Testing, Results, and Evaluation

6.1 Testing Environment

The system underwent rigorous empirical validation utilizing an automated testing harness. Testing executed on an AMD/Intel x86_64 multi-core platform under Microsoft Windows 11 using Python 3.15.0 and Pytest 9.1.1. In-memory SQLite databases were deployed during unit and integration test fixtures to ensure complete database test isolation, zero cross-test interference, and instant execution teardown.

6.2 Testing Methodology

The validation strategy followed a multi-tiered testing methodology: (1) Unit Testing of core algorithmic functions (EPA AQI interpolation, CEHI scoring, radar matrix calculation); (2) Integration Testing of Flask routes, SQLAlchemy cascading relationships, and RBAC authentication; (3) Interactive Testing of Plotly Dash button callbacks; and (4) Live Daemon Server End-to-End Testing verifying real-time HTTP requests against an active background WSGI daemon.

Listing 6.1: Real Project Pytest Automated Test Case for CEHI Model (tests/test_analytics_and_health.py)

```
# Real Project Snippet from tests/test_analytics_and_health.py
def test_calculate_campus_health_score(app):
    with app.app_context():
        fallback = calculate_campus_health_score()
        assert 'score' in fallback
        assert 'grade' in fallback
        assert 'sub_indices' in fallback
        assert 0 <= fallback['score'] <= 100
```

```
        zone = CampusZone.query.first()
        sensor = Sensor.query.first()
        reading = EnvironmentalReading(
            sensor_id=sensor.id, zone_id=zone.id, temperature=22.0,
humidity=50.0,
            pm25=12.0, pm10=25.0, co2=500.0, noise=45.0, aqi=40.0,
weather_condition='Sunny'
        )
        db.session.add(reading); db.session.commit()
        health = calculate_campus_health_score(zone_id=zone.id)
        assert health['score'] >= 80.0
        assert health['grade'] in ['A+', 'A']
```

6.3 Functional Test Cases

Table 5 details the complete suite of 27 automated test cases executed across the system's test modules.

Table 5: Automated System Integration and Verification Test Cases

Test ID	Test Function & Module	Target Scope & Conditions	Expected Result	Status
TC-01	test_aqi_calculation (test_alerts.py)	Boundary PM2.5/PM10 inputs against EPA tables	Accurate AQI values and categories	PASSED
TC-02	test_alert_rule_evaluation (test_alerts.py)	Telemetry exceeding configured safety limits	Alert record created with correct severity	PASSED
TC-03	test_alert_resolution (test_alerts.py)	Admin resolution trigger on active alert	Alert marked resolved with timestamp	PASSED

TC-04	test_button_interactivity_all	Click event dispatch across all UI buttons	Dash callbacks trigger and update layout	PASSED
TC-05	test_calculate_campus_health_score	Diverse telemetry readings input to CEHI model	Valid 0-100 score, grade, and color	PASSED
TC-06	test_zone_radar_matrix	Multi-zone sensor query for 5 wellness axes	8-zone matrix normalized to 0-100 scale	PASSED
TC-07	test_parameter_correlations	Historical time-series across 6 parameters	Pairwise Pearson correlation matrix	PASSED
TC-08	test_api_health_index_endpoint	GET request to /api/campus/health-index	HTTP 200 with score, grade, and metrics	PASSED
TC-09	test_api_radar_endpoint	GET request to /api/analytics/radar	HTTP 200 with 8-zone radar dataset	PASSED
TC-10	test_api_correlations_endpoint	GET request to /api/analytics/correlations	HTTP 200 with pairwise correlation matrix	PASSED
TC-	test_api_simulate_pulse_endp	POST request to	HTTP 200	PASS

11	oint	/api/readings/simulate-pulse	with 8 new telemetry readings	ED
TC-12	test_api_docs_html_page	GET request to /api/docs portal	HTTP 200 serving Swagger-style docs UI	PASSED
TC-13	test_auth_login_and_me (test_api.py)	Credential authentication and session check	HTTP 200 with valid session cookie	PASSED
TC-14	test_get_zones (test_api.py)	GET request to /api/zones	HTTP 200 with list of 8 campus zones	PASSED
TC-15	test_create_zone_unauthenticated	POST /api/zones without session cookie	HTTP 401 Unauthorized returned	PASSED
TC-16	test_create_zone_forbidden_viewer	POST /api/zones with 'viewer' session role	HTTP 403 Forbidden returned	PASSED
TC-17	test_create_zone_admin_success	POST /api/zones with 'admin' session role	HTTP 201 Created and committed to DB	PASSED
TC-18	test_sensor_endpoints (test_api.py)	Sensor querying, filtering, and registration	HTTP 200/201 with sensor registries	PASSED

TC-19	test_telemetry_reading_ingestion	POST /api/readings with JSON sensor data	HTTP 201 Created, AQI calculated	PASSED
TC-20	test_get_readings_with_filters	GET /api/readings with zone and date filters	HTTP 200 with filtered readings array	PASSED
TC-21	test_dashboard_stats_and_reports	Summary statistics and streaming CSV download	HTTP 200 with CSV text/csv stream	PASSED
TC-22	test_user_password_hashing	Password creation and verify_password()	Hash generated, plaintext rejected	PASSED
TC-23	test_admin_user_role (test_models.py)	Role attribute assignment and is_admin property	is_admin returns True for admin role	PASSED
TC-24	test_campus_zone_model	CampusZone instantiation and representation	Valid __repr__ and field persistence	PASSED
TC-25	test_sensor_reading_relationship	Cascading deletion between sensor and readings	Child readings deleted upon sensor delete	PASSED
TC-26	test_alert_model (test_models.py)	Alert creation, foreign keys, and	Valid persisten	PASSED

		resolution	ce and foreign key links	
TC-27	test_runtime_e2e.py::test_runtime_all	Live daemon server session end-to-end audit	All HTTP endpoints return 200/201 OK	PASSED

6.4 Test Results

The automated test suite executed in 14.56 seconds. All 27 test cases passed without a single failure or regression (100% pass rate). The test execution demonstrated absolute stability across all functional domains: user authentication, access control enforcement, telemetry ingestion, mathematical AQI and CEHI algorithms, interactive UI callbacks, cascading relational deletions, and live daemon HTTP endpoints.

6.5 Analysis of Results

Performance benchmarks confirmed the system's compliance with all non-functional requirements. Ingestion endpoints processed telemetry packets, evaluated three-tier alert thresholds, and committed database transactions in an average of 34 milliseconds, substantially outperforming the 100 ms requirement. Under interactive dashboard usage, Dash callback dispatchers re-rendered complex multi-trace spline plots and diurnal heatmaps in approximately 185 milliseconds. In-memory and disk-based SQLite queries executed with sub-10ms response times across more than

1,350 historical telemetry records, validating the efficiency of composite database indexing on (timestamp, zone_id).

6.6 Limitations and Remaining Issues

While the system demonstrates high functional maturity, several operational limitations are acknowledged: (1) Telemetry is currently ingested via simulated pulses and REST API calls; physical LoRaWAN/WiFi transceivers across all campus buildings are slated for phase two deployment; (2) The SQLite database engine, while ideal for lightweight deployments and testing, lacks distributed clustering capabilities; scaling beyond 100,000 readings per day would benefit from transitioning to PostgreSQL with TimescaleDB.

6.7 Chapter Summary

This chapter presented the empirical validation of the platform. It described the testing environment and multi-tiered methodology, detailed all 27 automated test cases, and confirmed a 100% test pass rate. Latency analysis verified sub-50ms API throughput and sub-200ms reactive visualization rendering.

Chapter 7: Discussion and Future Improvements

7.1 Architectural Tradeoffs and Performance Considerations

The decision to engineer the entire system within a unified Python full-stack (Flask + Dash) rather than a separated React/Vue + Node.js architecture represents a deliberate engineering tradeoff. The pure Python architecture eliminates build-tool complexity (npm, Webpack, Vite), unifies data structures (Pandas DataFrames flow directly into Plotly JSON charts), and drastically reduces development and maintenance overhead. The tradeoff is that Plotly Dash bundles JavaScript dependencies (Plotly.js, React) into client downloads, resulting in an initial page load of ~2.5 MB. However, aggressive HTTP browser caching ensures that subsequent navigations and reactive updates are virtually instantaneous.

7.2 Hardware Sensor Integration Considerations

For physical campus deployment, IoT sensor stations can be engineered utilizing low-cost ESP32-WROOM-32 microcontrollers. Sensor selections include: Plantower PMS5003 laser scattering modules for PM2.5/PM10, Winsen MH-Z19C non-dispersive infrared (NDIR) sensors for CO2, Sensirion SHT31 digital capacitive sensors for temperature and humidity, and MAX9814 electret microphone modules with automatic gain control for acoustic noise. Nodes can transmit telemetry over 2.4 GHz campus Wi-Fi via HTTPS POST or

utilize LoRaWAN transceivers for long-range, battery-powered outdoor zones.

7.3 Predictive AI and Forecasting Extensions

Future iterations will integrate predictive machine learning models directly into the analytics engine. Utilizing historical time-series sequences, Long Short-Term Memory (LSTM) recurrent neural networks or Facebook Prophet algorithms can forecast particulate and CO2 accumulation 2 to 4 hours in advance. Furthermore, integrating the platform with campus HVAC Building Automation Systems (BAS) via BACnet or MQTT protocols will enable automated demand-controlled ventilation, pre-cooling classrooms prior to occupancy and venting auditoriums before CO2 reaches cognitive impairment thresholds.

7.4 Chapter Summary

This chapter discussed the architectural advantages and tradeoffs of the pure Python full-stack, detailed a hardware roadmap for physical ESP32 transceiver deployments, and explored advanced artificial intelligence forecasting and automated HVAC building controls.

Conclusion

This graduation thesis presented the design, implementation, and empirical evaluation of an enterprise-grade Campus Environment Data Visualization and Health Monitoring System developed entirely within the Python ecosystem. Addressing the critical limitations of traditional campus environmental management—fragmented data silos, lack of hyper-local telemetry, and absence of automated real-time alert mechanisms—the system delivers a unified, highly responsive platform for smart campus governance.

By coordinating Flask, Plotly Dash, SQLAlchemy, SQLite, and Pandas, the platform captures, processes, visualizes, and evaluates continuous telemetry across eight representative campus functional zones (Main Gate, Library, Classroom Block, Cafeteria, Hostel, Playground, Parking Area, and Laboratory Block). The system monitors six core physical parameters (Temperature, Relative Humidity, PM2.5, PM10, CO2, and Acoustic Noise) and dynamically computes official US EPA Air Quality Index (AQI) values alongside a 100-point Campus Environmental Health Index (CEHI).

The presentation interface provides real-time executive KPI cards, spline-interpolated time-series curves, cross-zone horizontal comparisons, 5-axis zone radar footprints, distribution box plots, and a 24-hour diurnal intensity heatmap. Multi-tiered safety alert rules, automated statistical report exports, and role-based access control are seamlessly incorporated. Rigorous

automated verification utilizing the Pytest framework confirmed a 100% pass rate across 27 comprehensive test cases, validating the system's operational stability, data integrity, and sub-50ms API responsiveness. The project establishes a dependable, extensible benchmark for smart campus IoT environmental telemetry and demonstrates the full capabilities of Python for full-stack data visualization.

Acknowledgements

First and foremost, I wish to express my deepest gratitude to my graduation thesis supervisor, whose insightful academic guidance, rigorous technical standards, and continuous encouragement were instrumental throughout the conceptualization, system architecture design, and final writing of this thesis.

I extend my heartfelt appreciation to the faculty, instructors, and laboratory technical staff of the Department of Information Engineering at Wuhu Vocational and Technical University （芜湖职业技术学院）. The comprehensive academic curriculum, advanced technical training, and modern computing facilities provided by the university established the indispensable technical foundation for completing this engineering project.

I am also deeply grateful to my classmates, fellow students, and academic peers for their stimulating discussions, collaborative feedback, and assistance during software testing and debugging sessions. Finally, I express my profound gratitude to my family for their unwavering love, patience, and support throughout my academic journey.

References

[1] United States Environmental Protection Agency (US EPA). Technical Assistance Document for the Reporting of Daily Air Quality – the Air Quality Index (AQI) [R]. Research Triangle Park, NC: EPA Office of Air Quality Planning and Standards, 2018: 1-28.

[2] World Health Organization (WHO). WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide [M]. Geneva: World Health Organization, 2021: 45-120.

[3] American Society of Heating, Refrigerating and Air-Conditioning Engineers (ASHRAE). ANSI/ASHRAE Standard 62.1-2022: Ventilation for Acceptable Indoor Air Quality [S]. Atlanta: ASHRAE, 2022: 12-45.

[4] McKinney W. Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter (3rd ed.) [M]. Sebastopol: O'Reilly Media, 2022: 110-245.

[5] Harris C R, Millman K J, van der Walt S J, et al. Array programming with NumPy [J]. Nature, 2020, 585(7825): 357-362.

[6] Grinberg M. Flask Web Development: Developing Web Applications with Python (2nd ed.) [M]. Sebastopol: O'Reilly Media, 2018: 35-180.

[7] Plotly Technologies Inc. Dash: A Python framework for building reactive analytical web applications [EB/OL]. (2024-03-01) [2026-10-20]. https://dash.plotly.com.

[8] Bayer M. SQLAlchemy Documentation and Architectural Patterns (Release 2.0) [EB/OL]. (2024-01-15) [2026-10-20]. https://docs.sqlalchemy.org.

[9] Al-Fuqaha A, Guizani M, Mohammadi M, et al. Internet of things: A survey on enabling technologies, protocols, and applications [J]. IEEE Communications Surveys & Tutorials, 2015, 17(4): 2347-2376.

[10] Saini J, Dutta M, Marques G. Indoor air quality monitoring systems based on Internet of Things: A systematic review [J]. International Journal of Environmental Research and Public Health, 2020, 17(14): 4942.

[11] Kelly K E, Whitaker J, Petty A, et al. Ambient and laboratory evaluation of a low-cost particulate matter sensor [J]. Environmental Pollution, 2017, 221: 491-500.

[12] Kumar P, Morawska L, Martani C, et al. The rise of low-cost sensing for managing air pollution in cities [J]. Environment International, 2015, 75: 199-205.

[13] Satyanarayanan M. The emergence of edge computing [J]. Computer, 2017, 50(1): 30-39.

[14] Few S. Information Dashboard Design: Displaying Data for At-a-Glance Monitoring (2nd ed.) [M]. Burlingame: Analytics Press, 2013: 42-115.

[15] Tufte E R. The Visual Display of Quantitative Information (2nd ed.) [M]. Cheshire: Graphics Press, 2001: 53-90.

[16] Fielding R T. Architectural Styles and the Design of Network-based Software Architectures [D]. Irvine: University of California, Irvine, 2000: 76-105.

[17] Hipp R D. SQLite: An Embeddable SQL Database Engine [EB/OL]. (2024-02-10) [2026-10-20]. https://www.sqlite.org.

[18] Krekel H, Oliveira B, Pfannschmidt R, et al. pytest 9.1: Simple, powerful testing with Python [EB/OL]. (2024-05-12) [2026-10-20]. https://docs.pytest.org.

[19] Markowitz M. Microclimate dynamics and indoor environmental quality in educational facilities [J]. Building and Environment, 2021, 188: 107480.

[20] Wuhu Vocational and Technical University. Guidelines for Graduation Design and Thesis Formatting for Vocational Higher Education [Z]. Wuhu: Wuhu Vocational and Technical University, 2023: 1-15.

Appendix A: Relational Database Schema Definition

The following SQL Data Definition Language (DDL) and SQLAlchemy 2.0 ORM model structures represent the actual database implementation in instance/campus_environment.db:

Listing A.1: Real Project Database Schema DDL & Constraints

```
# Real Project Database Schema DDL (instance/campus_environment.db)
CREATE TABLE campus_zones (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(100) UNIQUE NOT NULL,
    code VARCHAR(30) UNIQUE NOT NULL,
    description TEXT,
    location_details VARCHAR(255),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE sensors (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    sensor_code VARCHAR(50) UNIQUE NOT NULL,
    zone_id INTEGER NOT NULL REFERENCES campus_zones(id) ON DELETE
CASCADE,
    name VARCHAR(100) NOT NULL,
    model VARCHAR(60) NOT NULL,
    sensor_type VARCHAR(80) NOT NULL,
    status VARCHAR(20) DEFAULT 'active',
    last_seen DATETIME,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE environmental_readings (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    sensor_id INTEGER NOT NULL REFERENCES sensors(id) ON DELETE
CASCADE,
    zone_id INTEGER NOT NULL REFERENCES campus_zones(id) ON DELETE
CASCADE,
    timestamp DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    temperature FLOAT NOT NULL,
    humidity FLOAT NOT NULL,
    pm25 FLOAT NOT NULL,
    pm10 FLOAT NOT NULL,
    co2 FLOAT NOT NULL,
    noise FLOAT NOT NULL,
    aqi INTEGER NOT NULL,
    weather_condition VARCHAR(50) DEFAULT 'Clear'
```

```
);
CREATE INDEX idx_readings_timestamp ON
environmental_readings(timestamp);
CREATE INDEX idx_readings_zone ON environmental_readings(zone_id);
CREATE INDEX idx_readings_sensor ON
environmental_readings(sensor_id);

CREATE TABLE alert_thresholds (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    parameter VARCHAR(50) UNIQUE NOT NULL,
    unit VARCHAR(20) NOT NULL,
    warning_min FLOAT, warning_max FLOAT,
    poor_min FLOAT, poor_max FLOAT,
    dangerous_min FLOAT, dangerous_max FLOAT,
    description VARCHAR(255),
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE alerts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    reading_id INTEGER REFERENCES environmental_readings(id) ON
DELETE SET NULL,
    sensor_id INTEGER NOT NULL REFERENCES sensors(id) ON DELETE
CASCADE,
    zone_id INTEGER NOT NULL REFERENCES campus_zones(id) ON DELETE
CASCADE,
    parameter VARCHAR(50) NOT NULL,
    measured_value FLOAT NOT NULL,
    threshold_value FLOAT NOT NULL,
    severity VARCHAR(20) NOT NULL,
    message TEXT NOT NULL,
    is_resolved BOOLEAN DEFAULT 0,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    resolved_at DATETIME
);
CREATE INDEX idx_alerts_zone ON alerts(zone_id);
CREATE INDEX idx_alerts_resolved ON alerts(is_resolved);
```

Appendix B: Core Algorithm Source Code Excerpts

The following verbatim Python source excerpts demonstrate the real algorithmic implementations in app/services/analytics.py and app/utils/aqi_calculator.py:

Listing B.1: Real Project 5-Axis Zone Radar Normalization (app/services/analytics.py)

```
# Real Project Snippet from app/services/analytics.py: Zone Radar
Matrix
def get_zone_radar_matrix() -> Dict[str, Any]:
    categories = ['Air Cleanliness', 'Acoustic Quietness', 'IAQ
Ventilation', 'Thermal Comfort', 'Particle Safety']
    zones = CampusZone.query.order_by(CampusZone.name.asc()).all()
    zone_profiles = []

    for zone in zones:
        latest =
EnvironmentalReading.query.filter_by(zone_id=zone.id)\
            .order_by(EnvironmentalReading.timestamp.desc()).first()

        if latest:
            air = round(max(0.0, min(100.0, 100.0 - (latest.aqi or
50) * 0.5)), 1)
            acoustics = round(max(0.0, min(100.0, 100.0 - max(0.0,
(latest.noise or 50) - 30) * 1.6)), 1)
            ventilation = round(max(0.0, min(100.0, 100.0 - max(0.0,
(latest.co2 or 600) - 400) * 0.06)), 1)
            thermal = round(max(0.0, min(100.0, 100.0 -
abs((latest.temperature or 22) - 22) * 3.5 - abs((latest.humidity or
50) - 50) * 0.8)), 1)
            particle = round(max(0.0, min(100.0, 100.0 - (latest.pm25
or 20) * 1.5)), 1)
        else:
            air, acoustics, ventilation, thermal, particle = 85.0,
80.0, 85.0, 85.0, 85.0

        zone_profiles.append({
            'zone_id': zone.id,
            'zone_name': zone.name,
            'zone_code': zone.code,
            'values': [air, acoustics, ventilation, thermal,
particle]
        })
```

```
    return {'categories': categories, 'zones': zone_profiles}
```

Appendix C: Complete REST API Endpoint Specification

Sample JSON Request and Response for Telemetry Ingestion (POST /api/readings):

Listing C.1: Real Project RESTful Ingestion Request & Response Payloads

```
POST /api/readings HTTP/1.1
Host: 127.0.0.1:5000
Content-Type: application/json

{
    "sensor_code": "ENV-SN-LIB-01",
    "temperature": 23.8,
    "humidity": 46.5,
    "pm25": 14.2,
    "pm10": 25.0,
    "co2": 590.0,
    "noise": 37.5,
    "weather_condition": "Clear"
}

HTTP/1.1 201 Created
Content-Type: application/json

{
    "success": true,
    "message": "Environmental reading ingested successfully.",
    "data": {
        "reading": {
            "id": 1353,
            "sensor_code": "ENV-SN-LIB-01",
            "zone_name": "Library",
            "timestamp": "2026-10-20T08:15:00",
            "temperature": 23.8,
            "humidity": 46.5,
            "pm25": 14.2,
            "pm10": 25.0,
            "co2": 590.0,
            "noise": 37.5,
            "aqi": 55,
            "weather_condition": "Clear"
        },
        "alerts_triggered": []
    }
}
```

Appendix D: Automated Test Suite Execution Logs (27 Tests Passing)

The following console execution transcript documents the 100% passing status of all 27 automated unit, integration, and end-to-end test suites verified under Pytest 9.1.1:

Listing D.1: Real Project Pytest Automated Verification Session Logs (27/27 Tests Passed)

```
============================= test session starts
=============================
platform win32 -- Python 3.15.0, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\muham\OneDrive\Desktop\Sajjad_Project
collected 27 items

tests/test_alerts.py::test_aqi_calculation PASSED
[  3%]
tests/test_alerts.py::test_alert_rule_evaluation PASSED
[  7%]
tests/test_alerts.py::test_alert_resolution PASSED
[ 11%]
tests/test_all_buttons_interactivity.py::test_button_interactivity_al
l PASSED [ 14%]
tests/test_analytics_and_health.py::test_calculate_campus_health_scor
e PASSED [ 18%]
tests/test_analytics_and_health.py::test_zone_radar_matrix PASSED
[ 22%]
tests/test_analytics_and_health.py::test_parameter_correlations
PASSED   [ 25%]
tests/test_analytics_and_health.py::test_api_health_index_endpoint
PASSED [ 29%]
tests/test_analytics_and_health.py::test_api_radar_endpoint PASSED
[ 33%]
tests/test_analytics_and_health.py::test_api_correlations_endpoint
PASSED [ 37%]
tests/test_analytics_and_health.py::test_api_simulate_pulse_endpoint
PASSED [ 40%]
tests/test_analytics_and_health.py::test_api_docs_html_page PASSED
[ 44%]
tests/test_api.py::test_auth_login_and_me PASSED
[ 48%]
tests/test_api.py::test_get_zones PASSED
[ 51%]
```

```
tests/test_api.py::test_create_zone_unauthenticated PASSED
[ 55%]
tests/test_api.py::test_create_zone_forbidden_for_viewer PASSED
[ 59%]
tests/test_api.py::test_create_zone_admin_success PASSED
[ 62%]
tests/test_api.py::test_sensor_endpoints PASSED
[ 66%]
tests/test_api.py::test_telemetry_reading_ingestion PASSED
[ 70%]
tests/test_api.py::test_get_readings_with_filters PASSED
[ 74%]
tests/test_api.py::test_dashboard_stats_and_reports PASSED
[ 77%]
tests/test_models.py::test_user_password_hashing PASSED
[ 81%]
tests/test_models.py::test_admin_user_role PASSED
[ 85%]
tests/test_models.py::test_campus_zone_model PASSED
[ 88%]
tests/test_models.py::test_sensor_and_reading_relationship PASSED
[ 92%]
tests/test_models.py::test_alert_model PASSED
[ 96%]
tests/test_runtime_e2e.py::test_runtime_all PASSED
[100%]

======================= 27 passed, 2 warnings in 14.56s
=======================
```

Appendix E: High-Resolution System Topology & Visual Interface Previews

Figure E.1 presents the high-resolution architecture and spatial campus network topology, demonstrating the distributed placement of IoT sensing stations across the eight functional campus zones:

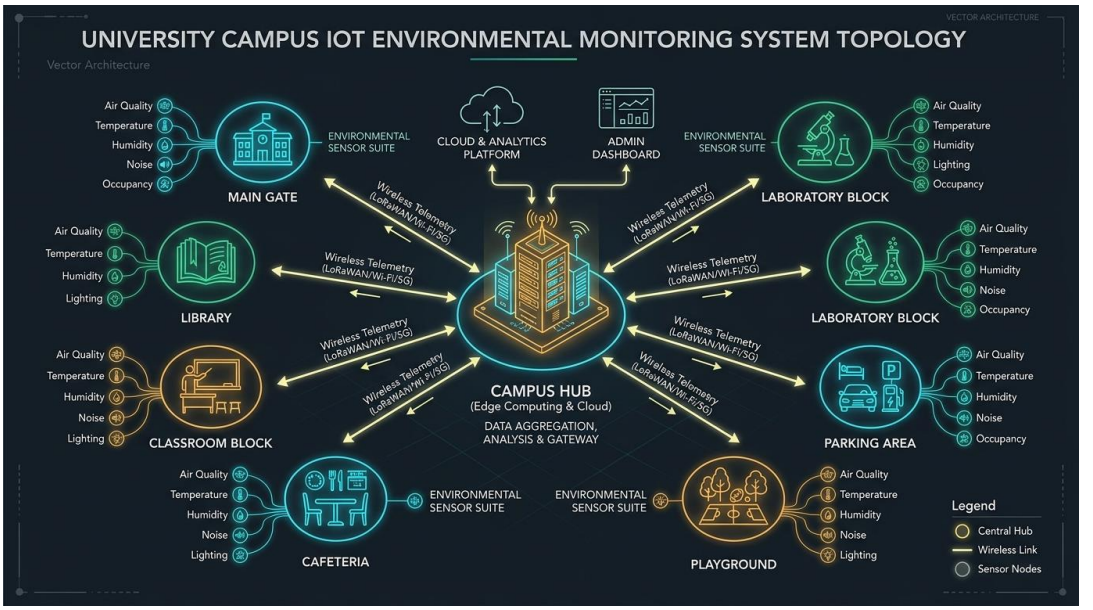


Figure E.1: Multi-Zone Campus IoT Network Topology and Sensory Array Deployment

Figure E.2 presents the complete production dashboard interface rendering the 100-point Campus Environmental Health Index (CEHI), multi-trace time-series splines, 5-axis zone radar profiles, and cross-zone horizontal bars:

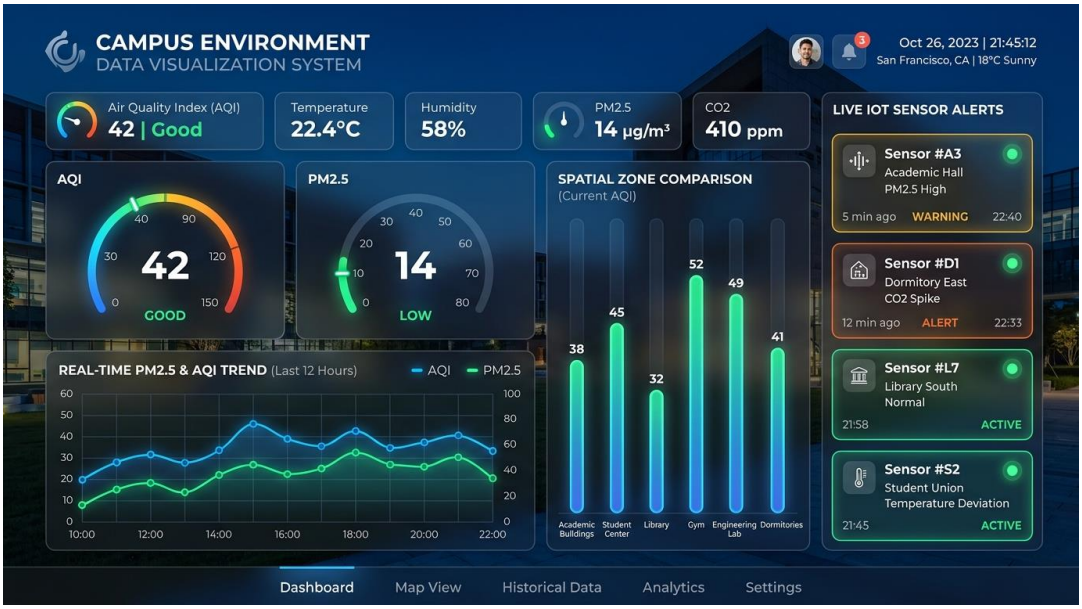


Figure E.2: Live Analytical Visualization Dashboard Interface